Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

RoadRank

Project Engineering

Year 4

Nasir Ahmad

Bachelor of Engineering (Honours) in Software and Electronic Engineering

Atlantic Technological University

2025/2026

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

_____

Acknowledgements

I would like to thank my supervisor Ben Kinsella for their guidance, support and valuable feedback throughout the development of this project.

Table of Contents

1 Summary 10
2 Poster 12
3 Introduction 13
3.1 Project Goal 14
3.2 Motivation 14
3.3 Layout of Report 14
4 Technologies Used 15
4.1 Hardware 15
4.1.1 Raspberry Pi & Pi Cam module 3 15
4.2 Software 17
4.2.1 Linux 17
4.2.2 Python 17
4.2.3 Google Colab and Jupyter Notebook 17
4.2.4 OpenCV 17
4.2.5 Flask 18
4.2.6 React, Next.js & Node.js 18
4.2.7 Express.js 18
4.2.8 MongoDB 18
4.2.9 AI Usage 18
5 Project Architecture 20
6 Project Plan 21
6.1 How the Plan Evolved 21
6.2 Project Tracking 22

6.3 Final Timeline 22
7 Machine Learning 23
7.1 Dataset 23
7.1.1 Driver Behavior Model 23
7.1.2 Facial Verification Model 23
7.2 Driver Behavior Model Network 25
7.2.1 Convolutional Neural Network (CNN) 25
7.2.2 Different Types of CNN 29
7.2.3 AlexNet 30
7.2.4 VGGNet 32
7.2.5 ResNet 35
7.3 Driver Behavior Model Result 36
7.4 Facial Verification Model Network 37
7.4.1 Preprocessing 37
7.4.2 Siamese Network 38
7.4.3 Training 41
7.5 Facial Verification Model Result 42
8 Website/Dashboard 45
8.1 Camera Server 45
8.1.1 State Machine 45
8.1.2 Frame Capture and Processing Loop 46
8.1.3 Identity Verification 47
8.1.4 Behavior Classification and Debouncing 48
8.1.5 HTTP Endpoints 49

8.2 Backend API Server 50
8.2.1 Authentication 50
8.2.2 Facial Setup and Frame Extraction 51
8.2.3 Behavior Logs, Alerts and Leaderboard 51
8.2.4 Data Model 52
8.3 Score Sync Worker 52
8.4 Frontend 53
8.4.1 Global State 53
8.4.2 Registration and Facial Setup 53
8.4.3 Login 54
8.4.4 Live Feed 54
8.4.5 Behavior Dashboard 55
8.4.6 Notifications 55
8.4.7 Leaderboard 55
9 Testing 56
9.1 User Testing 56
9.2 Performance Testing 56
9.3 Unit Tests 56
10 Ethics 59
10.1 Privacy and Biometric Data 59
10.2 Driver Consent and Third Parties 59
10.3 Bias in Machine Learning 60
10.4 Gamification and Behavioral Effects 60
10.5 Potential Misuse 60

11 Conclusion 61

12 Appendix 62

1 Summary

This project develops an intelligent behavior monitoring system that leverages machine learning to detect and track user behavior patterns. The primary goal was to create a comprehensive solution that combines facial verification with behavioral analysis to provide actionable insights and encourage positive habits through gamification.

The project encompasses a full-stack IoT architecture integrating hardware and software components. Key features include a facial verification model for user authentication, a behavior detection model for activity recognition, real-time behavior tracking with historical data visualization, a competitive leaderboard system, and an intuitive web interface for user interaction.

The development approach prioritized model validation before application integration. Initial efforts focused on training and testing the machine learning models in isolation using Google Colab and Jupyter Notebook to ensure accuracy and reliability. Once validated, these models were deployed within a production-ready application environment.

The technology stack combines modern web frameworks with IoT hardware. The frontend utilizes React for responsive user interfaces, while the backend leverages Next.js and Express.js for server-side functionality and API management. A Raspberry Pi equipped with a camera module serves as the edge computing device, running both the behavior detection models and the camera server for real-time video processing.

The completed application successfully tracks user behavior continuously, stores historical data for trend analysis, and displays comprehensive analytics through an interactive dashboard. Users are ranked on a leaderboard based on their behavioral metrics, fostering engagement and motivation. This system demonstrates the practical application of machine learning in IoT environments, proving that edge computing devices can effectively run sophisticated models while maintaining user privacy and system responsiveness.

2 Poster

Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University
Road Rank
By Nasir Ahmad
Beng.(Hons) in Software & Electronic Engineering
Inspired by Valeo
Project Summary
RoadRank is a machine learning IoT system that monitors drivers in real time using a camera-equipped Raspberry Pi 5. Two deep learning models run on-device: a custom VGGNet classifier that detects unsafe behaviours (texting, calling, turning, and more) and a Siamese neural network that enables passwordless facial recognition login. Each detection event is scored and stored in a MongoDB database, then surfaced through a Next.js web dashboard. Drivers can view their personal behaviour history, receive instant alerts when dangerous activity is detected, and compete against other drivers on a global safety leaderboard.
Website
Technologies Used
TensorFlow
Next.js
React
MongoDB
Express.js
openCV
Flask
Raspberry Pi Camera module 3
Raspberry Pi
Why?
Driver distraction is one of the leading causes of road accidents. Phones, fatigue, and inattention cause preventable harm every day, yet drivers rarely receive real-time feedback on their behavior. This project changes that, using computer vision to monitor driving habits and deliver feedback.
user
Camera feed
Raspberry Pi 5
Facial Verification
Behaviour Tracking
Camera server
sync server
Logged_data.json
Syncing data
Express Backend
Rest API
Behaviour alerts
Leaderboard
Facial frame storage
syncing score daemon
Architecture Diagram
Read/Write
Uploading data
Next.js Website
24/7 Live Feed
Behaviour data
Leaderboard
Notifications
Behaviour scoring
Polling Raspberry Pi
Display data
Database MongoDB
Users
username
email
password
Leaderboards
name
score
rank
Alerts
username
behavior
time
BehaviorData
username
behavior
score
Hardware
RPI camera module 3
Raspberry Pi

3 Introduction

Road safety remains a critical public concern in Ireland. In 2023, a total of 184 people lost their lives in 173 fatal collisions on Irish roads, a 19% increase compared to the 155 deaths recorded in 2022 [1]. While there was a modest improvement the following year, 174 people still died in 160 fatal collisions in 2024, with Dublin, Cork, Mayo, and Donegal accounting for 45% of all fatalities [2]. These figures paint a sobering picture of road safety in Ireland and highlight the urgent need for more effective monitoring and intervention.

The causes behind these tragedies are well documented. Speeding, intoxicated driving, fatigued driving, distracted driving, and the non-wearing of seatbelts are identified as key contributory factors to road deaths, with increases in such behaviors observed in the post-pandemic period [3]. Distraction at the wheel is a particularly persistent issue: in a single week alone, over 330 fixed-charge notices were issued for mobile phone use while driving, and 85 fines were handed out for failure to wear a seatbelt [4]. Drink driving compounds the problem further; RSA statistics show that almost one in eight Irish motorists admitted to driving after consuming alcohol in the past 12 months [5].

Young people are disproportionately affected by these dangers. The RSA has identified a notable increase in young people's deaths, with those aged 16 to 25 accounting for 27% of all road deaths in both 2023 and 2024 [6]. Meanwhile, Ireland’s broader road safety ambitions remain under pressure, the government's Road Safety Strategy 2021–2030 aims to reduce road deaths to 72 or fewer per year by 2030, as part of a long-term Vision Zero commitment to eliminate all road fatalities by 2050 [2].

It is within this context that a driver monitoring system (DMS) becomes not just an engineering challenge, but a matter of genuine public importance. Systems capable of detecting dangerous driver behaviors in real time — such as distraction, drowsiness, or inattention — have the potential to play a meaningful role in reducing collisions and saving lives on Irish roads.

3.1 Project Goal

The goal of the project was creating a Driving Monitoring System capable of detecting and classifying driver behaviors in real time using machine learning models. These models would then be used for creating a leaderboard that would rank people based on how safe they drive.

3.2 Motivation

The motivation of the project stemmed from my work placement at Valeo, where I worked with automotive camera systems designed to monitor driver behavior. These cameras used onboard software to track where the driver was looking and what they were doing at any given moment, primarily for safety purposes. Observing these technologies in a professional setting sparked my curiosity.

This curiosity became the driving force behind the project. I set out to build something similar from the ground up, with the dual goal of deepening my understanding of machine learning and exploring how trained models can be deployed in real world environments.

3.3 Layout of Report

This report is structured across 2 semesters of development. In this report I will first be reviewing similar existing technologies and the relevant background context, justifying the hardware and software choices made for the project. I will then present the project timeline and evaluate whether it was adhered to.

From there I move into the core technical details of semester 1, this includes research, the creation and sourcing of datasets, the training of the behavior model and facial verification model, and a detailed breakdown of the technologies used.

The second semester shifts focus to building a full-stack web application around those models. This covers the development of a camera server, the frontend and backend of the website, database integration using MongoDB, and the API layer built with Express.js, among other components.

4 Technologies Used

4.1 Hardware

For my project to function I needed a single board computer and a camera, during my initial research there were many single board computers some cheaper and some more expensive than the Raspberry Pi 5. I chose the Raspberry Pi 5 over its competition for the following reasons.

4.1.1 Raspberry Pi & Pi Cam module 3

At the time of purchase the Raspberry Pi 5 (RPI) was a low cost, single board computer developed by the Raspberry Pi Foundation, originally designed to promote computer science education but since adopted widely in research, industry, and IoT development. The Raspberry Pi 5 features an ARM CPU and can be configured with 4, 8 or 16 gigabytes of RAM.

The Raspberry Pi 5 was selected as the edge computing device for several reasons. It has a large and well-documented community, meaning support, tutorial, and libraries are readily available, significantly reducing development time. The RPI runs a full Linux-based operating system, meaning frameworks such as TensorFlow, OpenCV, and Python are natively supported with minimal configuration. The RPI has also been demonstrated in AI workloads and real-time monitoring systems, establishing it as a practical platform for running machine learning models at the edge.

Figure 1. Raspberry Pi 5

The Pi Camera Module 3 pairs directly with the board via the MIPI CSI interface, offering high-quality image capture with low latency and official driver support through the `libcamera` stack, this makes it straightforward to integrate into a real time video processing pipeline. While alternatives such as the NVIDIA Jetson series offer greater raw inference performance, they come with significantly higher power consumption and a steeper learning curve that would have added unnecessary complexity [7].

Figure 2. RPI Cam Module 3

For a student-built system focused on demonstrating real world machine learning deployment the Raspberry Pi 5 and Camera Module 3 represented the most practical and appropriate choice.

4.2 Software

4.2.1 Linux

Linux served as the operating system for the Raspberry Pi throughout this project, specifically Raspberry Pi OS, a Debian-based distribution maintained by the Raspberry Pi Foundation [8]. Linux was chosen because it is the standard environment for embedded and IoT development, offering full control over system processes, networking, and hardware interfaces such as the camera. All the key software used in this project Python, OpenCV, Flask, TensorFlow run natively on Linux, and the operating system's open-source nature means that most dependencies can be installed quickly through standard package managers.

4.2.2 Python

Python was the primary programming language used throughout the project, particularly for model training and backend processing. It has become the dominant language in the machine learning and computer vision space due to its clean syntax, extensive library ecosystem, and strong community support [9]. Its versatility also meant that the same language could be used across the Raspberry Pi camera server and the Flask backend, keeping the development stack consistent.

4.2.3 Google Colab and Jupyter Notebook

Google Colab and Jupyter Notebook are development environments that are used for writing and running code in notebook style documents, especially popular for Python and machine learning. Google Colab is a cloud-based notebook environment provided by Google [10]. It offers free access to GPU-accelerated computing. Jupyter Notebook is run locally or on your own server. These were used for training the behavior model and facial verification model.

4.2.4 OpenCV

OpenCV (Open-Source Computer Vision Library) is a widely used open-source library for real-time computer vision tasks. It provides tools for image capture, frame processing, object detection, and video stream management [11]. In this project, OpenCV was used to handle the live video feed from the Raspberry Pi Camera Module, process individual frames before passing them to the machine learning models and manage the real-time inference pipeline. Its tight

integration with Python and its efficiency on resource-constrained devices made it a practical fit for running on the Raspberry Pi.

4.2.5 Flask

Flask is a lightweight Python web framework used to build the camera server component of this project. Unlike heavier frameworks Flask is minimal by design, making it straightforward to set up a simple HTTP server that can stream video frames and expose endpoints for the frontend to consume. Its simplicity made it the right tool for bridging the RPI hardware and the wider web application. [12]

4.2.6 React, Next.js & Node.js

React is a component-based JavaScript library developed by Meta for building interactive user interfaces. [13]

Next.js is a framework built on top of React, It provides a structured way to build full-stack web applications within a single project, handling both the frontend rendering and backend API routes. [14]

Node.js is a JavaScript runtime that powers both Next.js on the server side and Express.js API Layer, allowing JavaScript to be run outside of the browser. [15]

4.2.7 Express.js

Express.js is a minimal and flexible Node.js web framework used to build the backend API for this project. It handles incoming HTTP requests from the frontend and communicates with the database. [16]

4.2.8 MongoDB

MongoDB is a NoSQL document-oriented database that stores data in a flexible, JSON-like format. It is commonly used for web applications because it is fast, scalable and works well with JavaScript tools like Node.js and Express.js. [17]

4.2.9 AI Usage

Generative AI tools were used during the development of this project in a supporting role rather than as a substitute for the work itself. ChatGPT and Claude were used to explain

unfamiliar concepts when first researching convolutional architectures and the Siamese network, to suggest alternative phrasings while drafting sections of this report, and to act as a sounding board when debugging issues such as the thread-coordination logic in the camera server and the ffmpeg pipeline used during facial setup. AI was used in the Front-End for CSS and general styling of the website.

5 Project Architecture

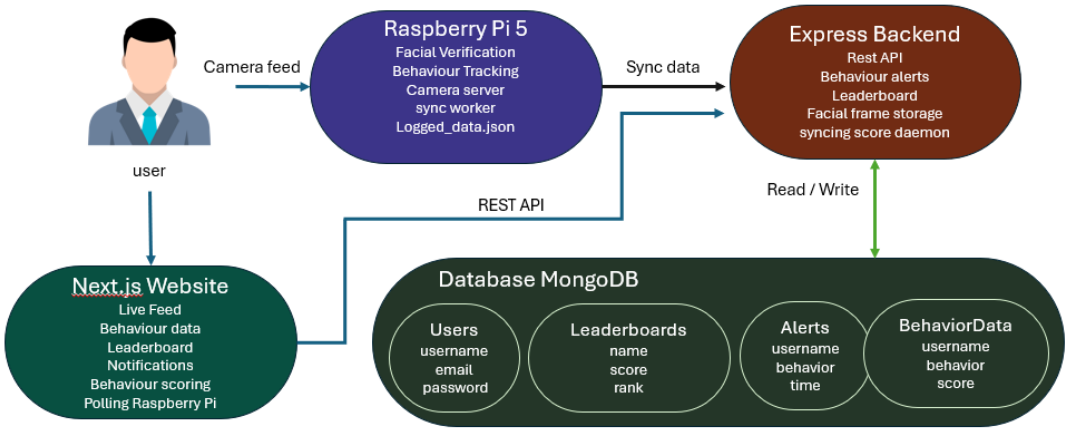

Figure 3. Architecture Diagram

6 Project Plan

The original proposal for this project, ML model for Driver Analysis, set out a two-semester delivery split. Semester 1 was scoped around research, dataset collection and an initial trained model, with a working prototype to be shown at the Christmas demo. Semester 2 was to focus on validating and improving that model, building a live alerting system, creating a web dashboard to display the recorded data, and producing the deliverables required at the end of the year i.e. the final report, the poster, the video, and the summer demo. The proposed system was framed as a Machine Learning Internet-of-Things project, with a Raspberry Pi 5 running a model that would recognize distracted-driving cues such as facial expressions, head position and (time permitting) hand position, with all alert data persisted to MongoDB.

6.1 How the Plan Evolved

The high-level structure of the proposal held throughout the year, but the scope of each semester shifted in response to what proved to be tractable and useful. In Semester 1 the focus on facial expressions, head position and hand position was narrowed to a single, well-defined classification problem: a five-class driver-behavior classifier (safe driving, turning, talking, texting and other) trained on the REVITSONE Kaggle dataset. This change was made early once it became clear that fusing multiple cue types within one model would have demanded a much larger custom dataset than was realistic to collect in a single semester. Three convolutional architectures; AlexNet, VGGNet and ResNet were trained and compared, and VGGNet was selected for deployment after it reached 95.44% test accuracy.

In Semester 2 a new component was added that was not in the original proposal: a facial verification stage based on a Siamese neural network. This was introduced once the gamified scoring system began to take shape in the Web App, because it became clear that scores tied to a username were only meaningful if the system could confirm the person sat in front of the camera was the registered driver. The verification model and the behavior classifier were then combined inside a state machine on the Pi (unverified -> verifying -> verified) so that scoring only began once the driver had been authenticated. The dashboard work expanded similarly.

The proposal mentioned a single web page showing recorded data; in practice this grew into a Next.js front-end with login, registration, facial-setup, live feed, score graphs, notifications and a leaderboard, backed by an Express REST API and a MongoDB Atlas cluster.

6.2 Project Tracking

Progress was tracked week by week throughout both semesters. I used Obsidian as a working journal and uploaded what I did each week to a Microsoft OneNote notebook, with one page per week containing notes, decisions and links to artefacts. Jira was used to track sprints and progress. Two-week sprints were used for the development phases. Source control was handled through Git, with the Raspberry Pi code, the Google Colab notebooks, the Express backend and the Next.js front-end each kept in their own directory of the project repository. Supervisor meetings with Mr. Kinsella were held regularly, and OneNote was used as the agenda for those meetings.

6.3 Final Timeline

By the end of Semester 1 the dataset had been collected and cleaned, the three candidate models had been trained and benchmarked, and a working VGGNet classifier was running on the Pi for the Christmas demo. Semester 2 delivered the remaining items in the proposal, the live alerting system, the dashboard, the poster, the final report and the summer demo video together with the Siamese verification model, the state machine, the gamified scoring system and the leaderboard, none of which were in the original scope. The only proposed element that was deliberately dropped was hand-position tracking, which was made redundant once the five-class behavior classifier was shown to capture the same distraction signals (texting and talking on the phone) more reliably.

7 Machine Learning

7.1 Dataset

7.1.1 Driver Behavior Model

The primary dataset used for training the driver behavior model was the REVITSONE 5-Class Driver Behavior Dataset, sourced from Kaggle. This dataset contained over 10,000 RGB images captured using a dashboard mounted camera during driving sessions, covering a range of real-world conditions including varied driver demographics such as age and gender, as well as different lighting conditions spanning both daytime and nighttime scenarios. The dataset is organized into five labelled classes, which are: safe_driving, talking_phone, texting_phone, turning and other_activities. Its open access licence made it well suited for this project.

The dataset was stored in Google Drive from where it was mounted directly into the Google Colab training environment. Before training a preprocessing step was applied to remove any corrupted or blank images from the dataset. This was done by verifying each image's integrity using PIL and checking for near-zero-pixel variance, a sign that an image contains no useful visual information. Any image that failed these checks was removed from the dataset before training.

To supplement this dataset and improve the model's ability to recognize behavior from the specific camera angle and environment used in the project, additional images were captured personally using the Raspberry Pi Camera Module 3. These custom images were taken to match the real deployment conditions. The Albumentations python library was used to augment these images by applying transformations such as lighting adjustments, noise injection, and contrast variation, artificially expanding the volume of training data.

By incorporating personally captured images the model was exposed to samples much closer to what it would encounter in practice, improving its ability to generalise to the actual use case.

7.1.2 Facial Verification Model

Unlike the behavior model which used a publicly available Kaggle dataset, the facial verification model required a completely custom dataset built from scratch. This was necessary because the

model needs to learn to recognize a specific individual. In this case the developer, so generic facial datasets would not serve the purpose. Three categories of images were collected and stored in dedicated folders:

- Anchor images: reference photos of the target person taken under normal conditions
- Positive images: additional photos of the same person, used to form matching pairs during training
- Negative images: photos of different people, used to form non-matching pairs

The images were captured directly using the Raspberry Pi Camera Module via the `picamera2` library, which allowed photos to be taken locally on the device under realistic lighting and framing conditions. Each image was assigned a unique filename using Python's `uuid` library to avoid naming conflicts. The folder structure was set up as follows:

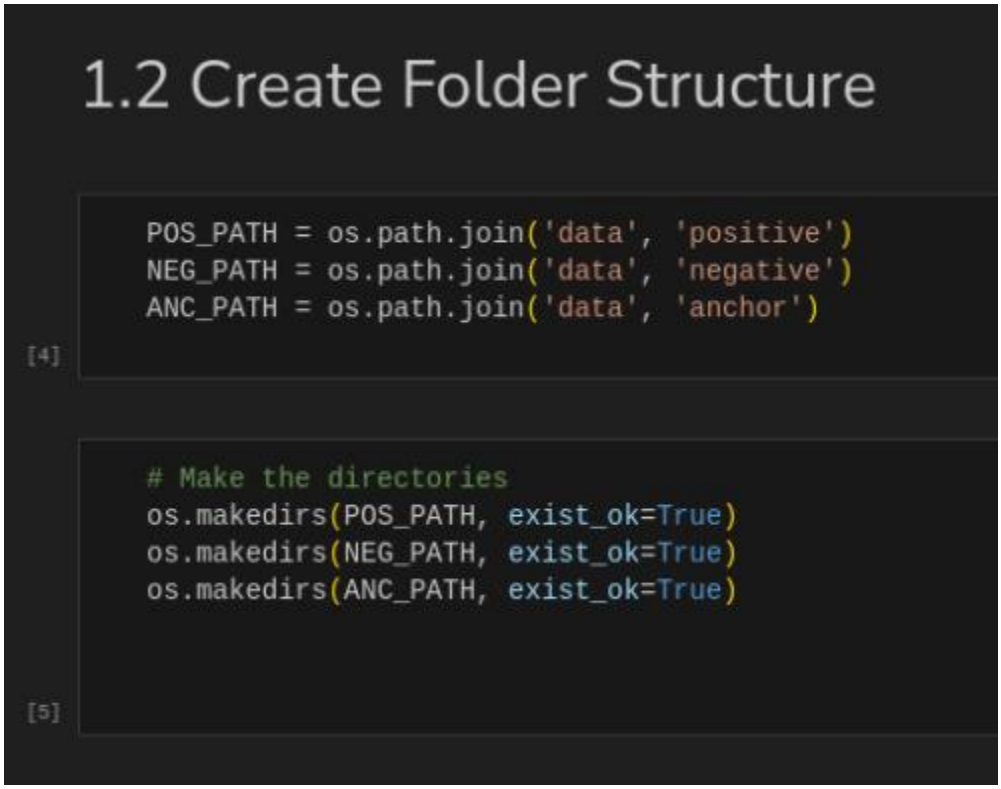

Figure 4. Siamese Dataset Folder Structure

Up to 300 images were taken from each category. Once collected, the dataset was structured into matched pairs, anchor/positive pairs labelled 1 (same person) and anchor/negative pairs labelled 0 (different person) and combined into a single TensorFlow dataset:

```
anchor = tf.data.Dataset.list_files(ANC_PATH+'/*.jpg').take(300)
positive = tf.data.Dataset.list_files(POS_PATH+'/*.jpg').take(300)
negative = tf.data.Dataset.list_files(NEG_PATH+'/*.jpg').take(300)
```

```
#zips dataset and creates labelled dataset for positive images filled with 1
#zipping allows us to iterate through all at same time
positives = tf.data.Dataset.zip((anchor, positive, tf.data.Dataset.from_tensor_slices(tf.ones(len(anchor)))))
#zips dataset and creates labelled dataset for negative images filled with 0
negatives = tf.data.Dataset.zip((anchor, negative, tf.data.Dataset.from_tensor_slices(tf.zeros(len(anchor)))))
#joining positives and negatives together to create one dataset
data = positives.concatenate(negatives)
[15]
```

Figure 5. Creating a Labelled Dataset

7.2 Driver Behavior Model Network

For the Driver Behavior Model, I tried 3 Convolutional Neural Networks. To train these different networks I used Google Colab.

7.2.1 Convolutional Neural Network (CNN)

A CNN is a type of neural network designed to process data that has spatial or sequential structure e.g. images. The key insight behind CNNs is that you shouldn't need to learn separate features for every possible position in an image. Instead, you learn a small kernel (filter) and slide it across the entire input, detecting the same feature wherever it appears. This is called weight sharing [18].

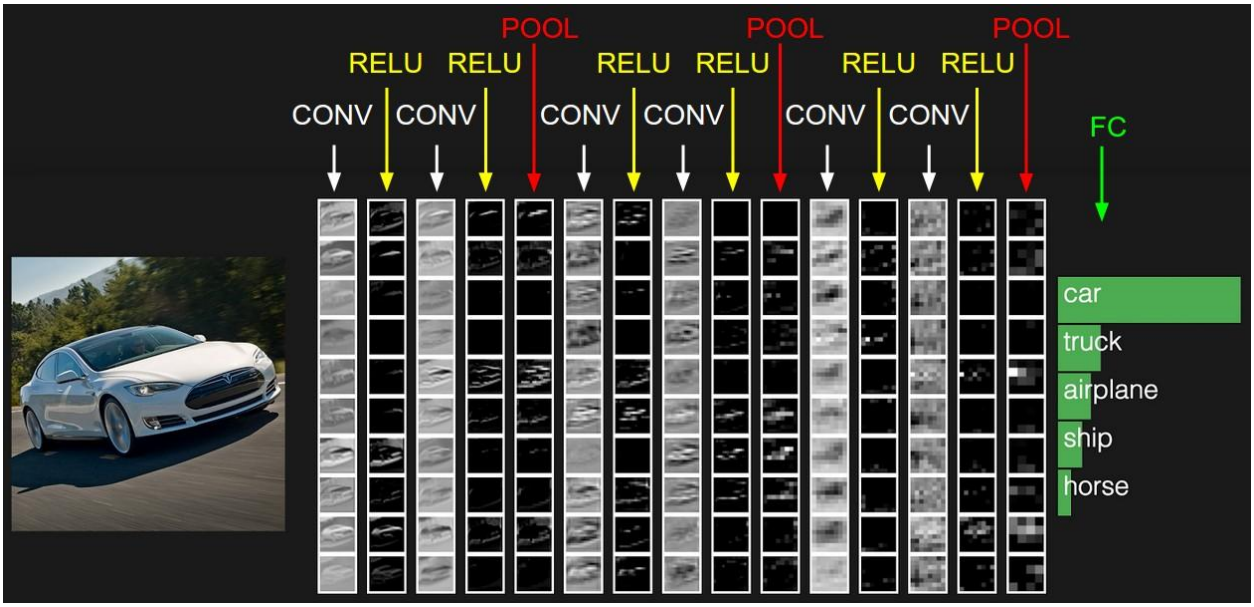

Figure 6. CNN Core Layer [19]

7.2.1.1 Convolutional Layer

This is the heart of the CNN. A small matrix called a kernel, typically 3x3 or 5x5 pixels slides across the input image. At each position, it computes a dot product between the filter weights and the local patch of pixels it's sitting on. This produces a single number. As the filter slides across the entire image (this sliding is called convolution), it produces a 2D grid of numbers called a feature map.

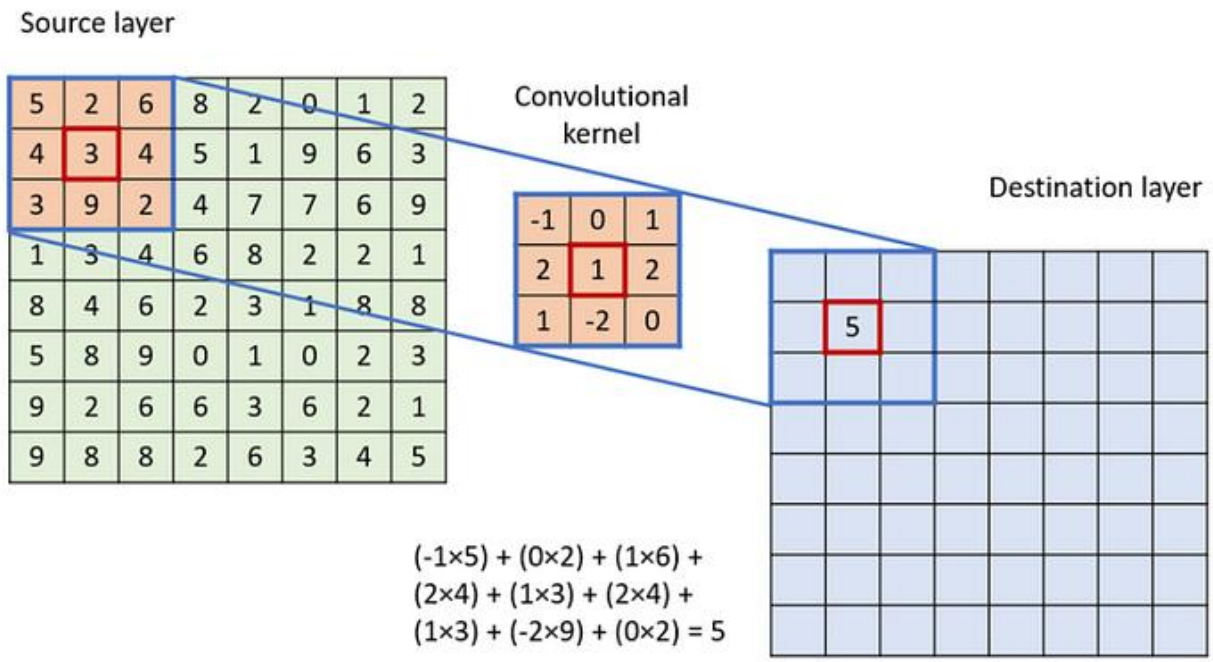

Figure 7. Convolution [20]

A single filter detects one specific pattern e.g. horizontal edge or a colour gradient. In practice, a convolutional layer uses many filters in parallel (32, 64, 128, etc.), each learning to detect a different feature. The output is a stack of feature maps, one per filter. There are some important parameters, these are:

Stride: How many pixels the filter moves at each step. A stride of 2 halves the spatial dimensions. [20]

Padding: Adding a border of zeros around the input so that the filter can reach the edges. Zero-padding keeps the output the same size as the input. No padding shrinks the output. [21]

Early convolutional layers tend to learn low level features (edges, texture, colours). Deeper layers combine those into higher level features (eyes, wheels).

7.2.1.2 Activation Layer

After convolution, you pass the feature map through a non-linearity. Without this step, stacking multiple convolutional layers would just collapse into one big linear operation.

Activation layers are not technically “layers” because no parameters/weights are learned inside an activation layer. They are sometimes omitted from network architecture diagrams because of this. [21]

The most common choice is ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Every negative value in the feature map becomes 0, positive number pass through unchanged. It’s fast to compute, doesn’t suffer from vanishing gradients as badly as older functions like sigmoid and works extremely well in practice. [22]

7.2.1.3 Pooling Layer

Pooling reduces the spatial dimensions of the feature maps. This does two things; it compresses the representation (less overall computation) and introduces a degree of translation invariance (can identify an object regardless of its location in an image).

Max Pooling is the most common. Take a 2x2 window, slide it across the feature map with stride 2, and keep only the maximum value in each window. The result is a feature map half the size in each spatial dimension, but the essential details remain intact. [23]

7.2.1.4 Fully Connected Layer

The Fully Connected Layer can also be called the Dense layer. After several rounds of convolution and pooling, the spatial structure has been compressed and abstracted into a rich set of features. Now you need to make an actual prediction. The feature maps are flattened into a 1D vector and passed through one or more fully connected layers; these are the same dense layers you’d find in a regular neural network. Every neuron connects to every value in the previous layer.

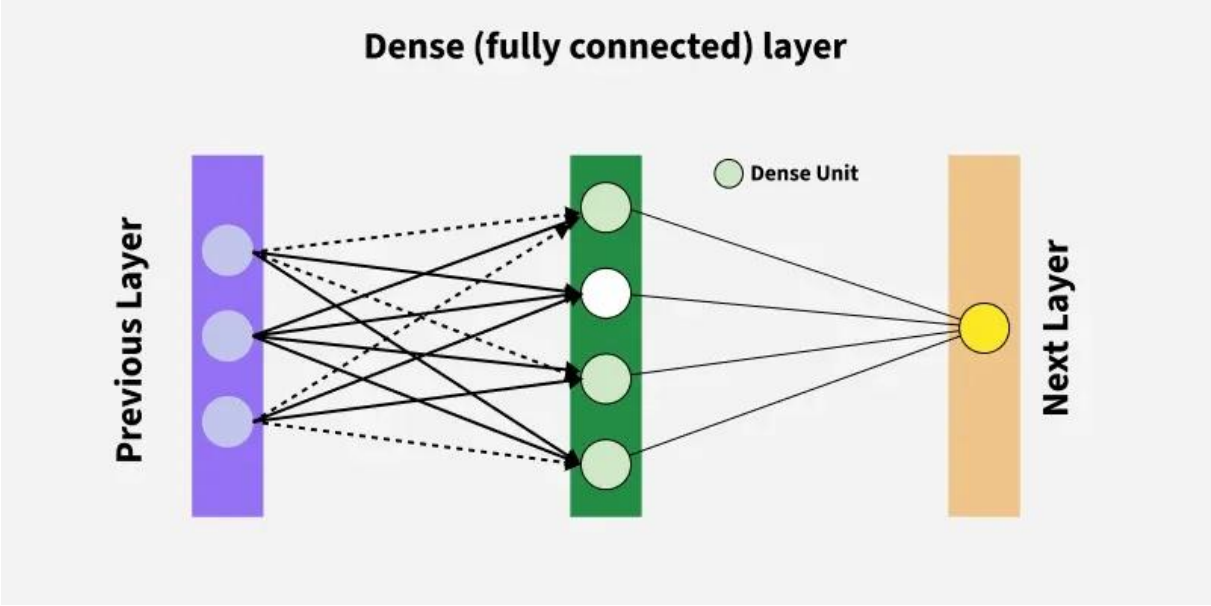

Figure 8. Fully Connected Layer [24]

7.2.1.5 Dropout

Dropout is a regularization technique used to reduce overfitting and improve test accuracy, sometimes at the cost of lower training accuracy. During training, for each mini batch, a dropout layer randomly disables connections from the previous layer with probability p, preventing the network from relying too heavily on specific neurons. [21]

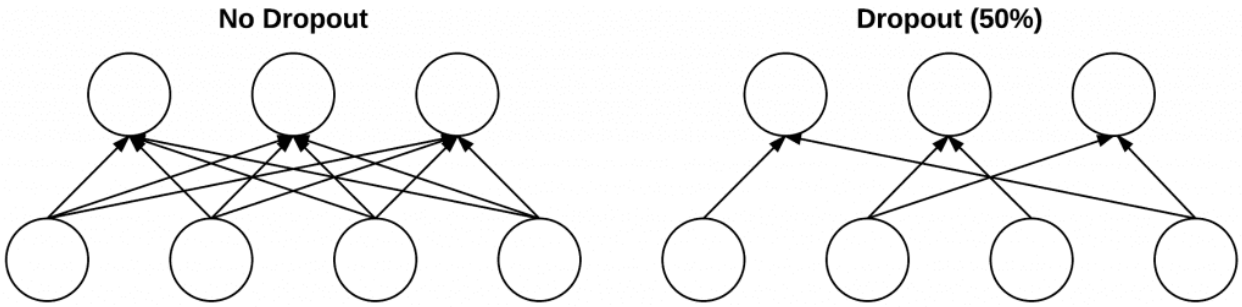

Figure 9. Two Layers of Neural Network [21]

7.2.2 Different Types of CNN

In this project I focused on 3 different types of CNN AlexNet, VGGNet and ResNet. These 3 Networks were trained in Google Colab.

7.2.3 AlexNet

AlexNet was one of the first deep convolutional neural networks to achieve breakthrough performance in image classification, winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 by a significant margin. It demonstrated that deep CNNs trained on GPUs could dramatically outperform traditional computer vision methods, and it is widely credited with igniting the modern deep learning era.

The network consists of 8 learnable layers, 5 convolutional layers and 3 fully connected layers. It introduced several key techniques that became standard in the field, including ReLU activations, dropout regularization and local response normalization.

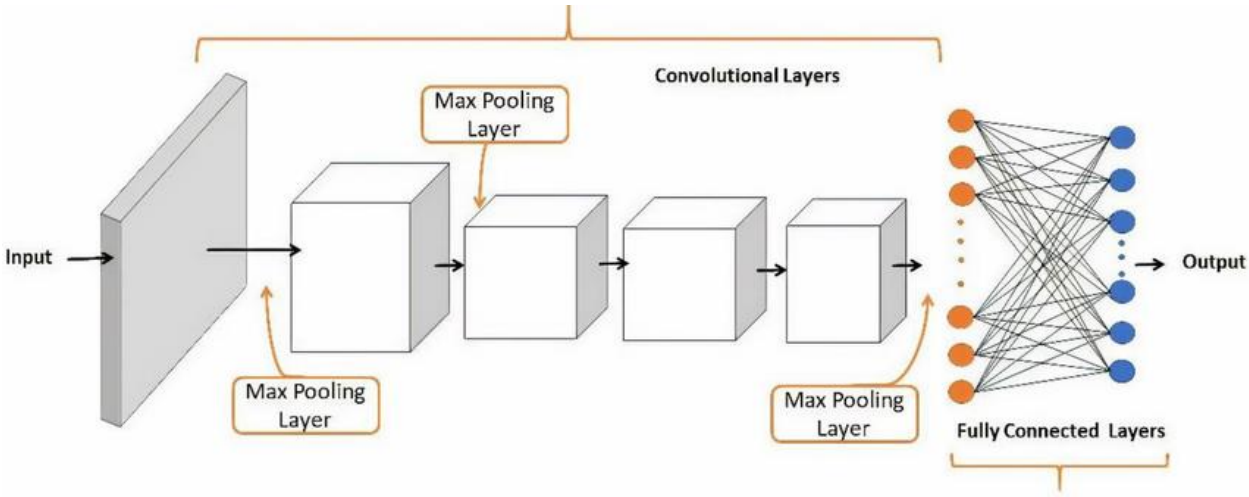

Figure 10. AlexNet CNN Architecture

In this project, AlexNet was implemented as the baseline model. The architecture takes a 240x240x3 input and passes it through a sequence of convolutional layers with progressively increasing filter counts (96 -> 256 -> 384 -> 384 -> 256), each followed by a max pooling where appropriate, before flattening and passing through two dense layers of 4096 units with 0.5 dropout. The final output layer uses softmax activation to produce a probability distribution across the five behavior classes.

```
#defining the Network
def AlexNet():
  inp = layers.Input((240, 240, 3))
  x = layers.Conv2D(96, 11, 4, activation='relu')(inp)
  x = layers.BatchNormalization()(x)
  x = layers.MaxPooling2D(3, 2)(x)
  x = layers.Conv2D(256, 5, 1, activation='relu')(x)
  x = layers.BatchNormalization()(x)
  x = layers.MaxPooling2D(3, 2)(x)
  x = layers.Conv2D(384, 3, 1, activation='relu')(x)
  x = layers.Conv2D(384, 3, 1, activation='relu')(x)
  x = layers.Conv2D(256, 3, 1, activation='relu')(x)
  x = layers.MaxPooling2D(3, 2)(x)
  x = layers.Flatten()(x)
  x = layers.Dense(4096, activation='relu')(x)
  x = layers.Dropout(0.5)(x)
  x = layers.Dense(4096, activation='relu')(x)
  x = layers.Dropout(0.5)(x)
  x = layers.Dense(5, activation='softmax')(x)

  #creating the model(inputs,outputs)
  model_Alex = models.Model(inputs = inp, outputs = x)

  return model_Alex

model_Alex = AlexNet()
```

Figure 11. AlexNet Architecture Code

Once the model was defined, it was compiled using the Adam optimizer and binary cross-entropy loss, then trained with a checkpoint to preserve the best weights:

```
model_Alex.compile(loss=BinaryCrossentropy(),
                   optimizer=Adam(learning_rate=0.001), metrics=['accuracy'])
```

```
#Create a callback that saves the models weights
cp_callback = keras.callbacks.ModelCheckpoint(filepath=checkpoints_path,
                                              save_weights_only = True,
                                              verbose = 1)

#Train model with the new callback
Alex_model = model_Alex.fit(trainDataset,
                            epochs=20,
                            validation_data=validDataset,
                            callbacks=[cp_callback])
```

Figure 12. AlexNet Training Code

7.2.4 VGGNet

VGGNet is a family of convolutional neural network architectures developed by the Visual Geometry Group at the University of Oxford.

Its key contribution was demonstrating that network depth, using very small 3x3 convolutional filters consistently throughout was a critical factor in achieving strong classification performance. Rather than using large filters like AlexNet's 11x11 first layer, VGGNet stacks multiple small convolutions, which achieves the same effective receptive field while introducing more non-linearity. The most common versions are VGG-16 and VGG-19, which feature 16 and 19 weight layers. It placed 2nd in the ImageNet challenge in 2014 and influenced future architecture designs such as ResNet.

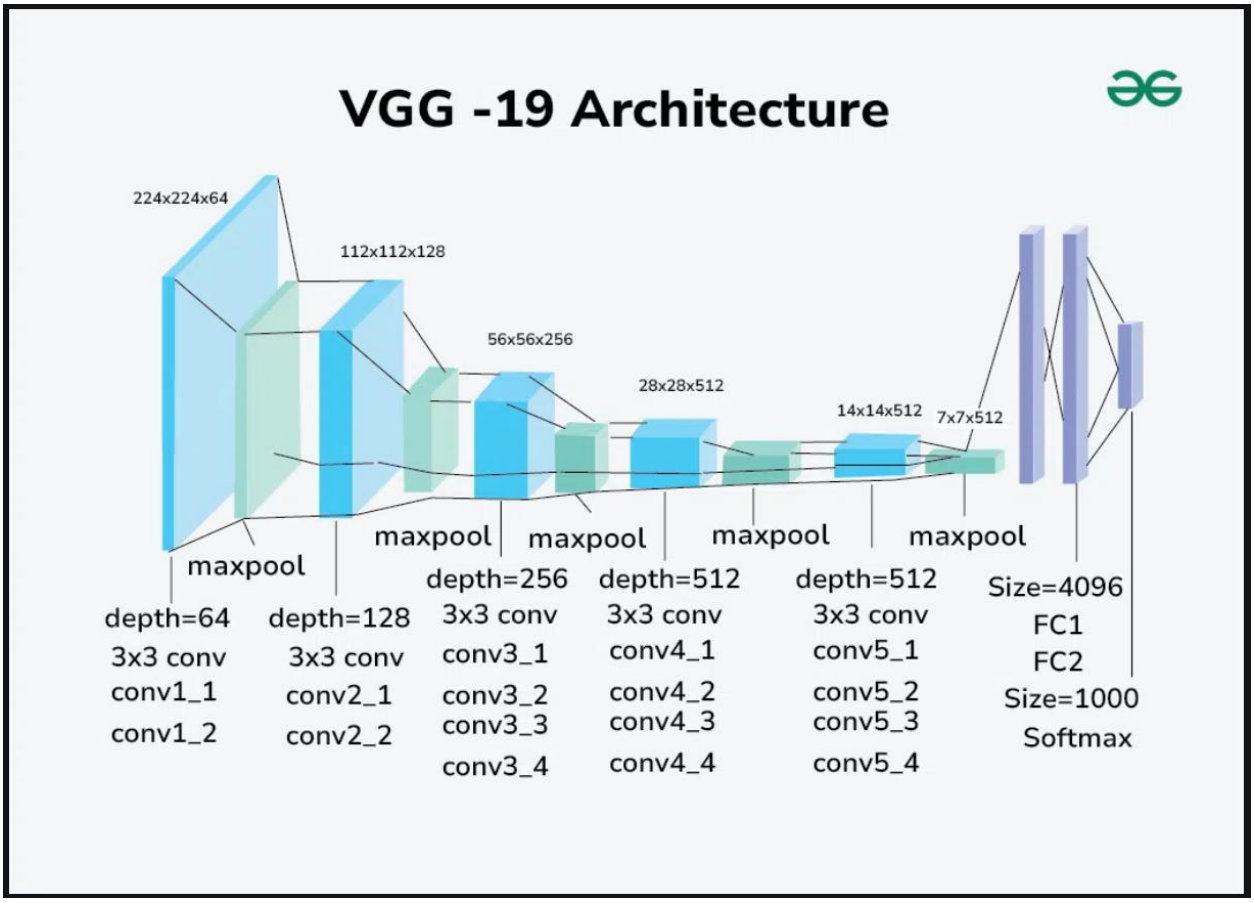

Figure 13. VGGNet Architecture

The VGGNet implementation in this project follows the core design principle of stacked 3x3 convolutions grouped into blocks of increasing filter depth (64 -> 128 -> 256 -> 512), each block followed by batch normalization and max pooling. After flattening two fully connected layers of 4096 units with dropout are used before the 5 class softmax output.

```
def VGGNet():
    inp = layers.Input((240, 240, 3))
    x = layers.Conv2D(64, 3, 1, activation='relu')(inp)
    x = layers.Conv2D(64, 3, 1, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2, 2)(x)
    x = layers.Conv2D(128, 3, 1, activation='relu')(x)
    x = layers.Conv2D(128, 3, 1, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2, 2)(x)
    x = layers.Conv2D(256, 3, 1, activation='relu')(x)
    x = layers.Conv2D(256, 3, 1, activation='relu')(x)
    x = layers.Conv2D(256, 3, 1, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2, 2)(x)
    x = layers.Conv2D(512, 3, 1, activation='relu')(x)
    x = layers.Conv2D(512, 3, 1, activation='relu')(x)
    x = layers.Conv2D(512, 3, 1, activation='relu')(x)
    x = layers.MaxPooling2D(2, 2)(x)
    x = layers.Flatten()(x)
    x = layers.Dense(4096, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(4096, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(5, activation='softmax')(x)

    model_VGG = models.Model(inputs=inp, outputs=x)

    return model_VGG

model_VGG = VGGNet()
```

Figure 14. VGGNet Architecture Code

The model was compiled and trained in the same manner as AlexNet, with its own dedicated checkpoint path:

```
#Create a callback that saves the models weights
cp_callback = keras.callbacks.ModelCheckpoint(filepath=checkpoints_path_VGG,
                                              save_weights_only = True,
                                              verbose = 1)

#Train model with the new callback
VGGNet = model_VGG.fit(trainDataset,
                       epochs=20,
                       validation_data=validDataset,
                       callbacks=[cp_callback])
```

Figure 15. VGGNet Training Code

7.2.5 ResNet

ResNet (Residual Network) was introduced by Microsoft Research in 2015 and won the ImageNet competition that year. Its central innovation was the residual connection also called skip connections, which allows the output of one layer to bypass one or more intermediate layers and be added directly to the output further down the network. This addressed the vanishing gradient problem that had made it difficult to train very deep networks, enabling architectures with dozens or even hundreds of layers to be trained effectively.

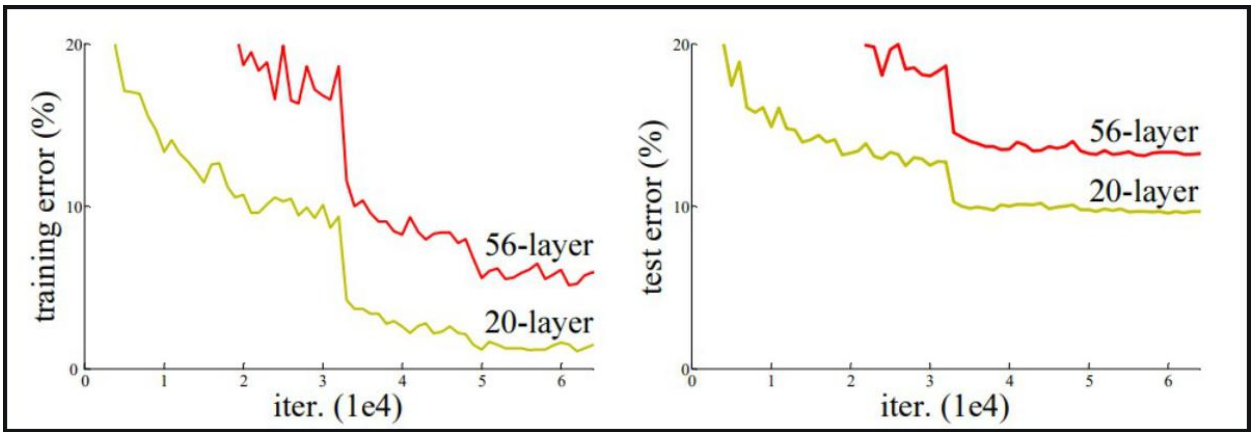

Figure 16. 20-layer vs 56-layer CNN [25]

As you can see from the above plot that a 56-layer CNN gives more error rate on both training and testing. This is the vanishing gradient problem.

The ResNet34 implementation in this project follows a four-stage structure with convolutional blocks of increasing filter sizes (64 -> 128 -> 256 -> 512), using 3x3 convolutions and batch normalization throughout. An initial 7x7 convolution with stride 2 is used to downsample the

input before the residual stages begin. Each stage doubles the filter count while reducing spatial resolution through strided convolutions. The final layers follow the same pattern as the other architectures: a flatten followed by two dense layers of 4096 units with dropout and a 5-class softmax output.

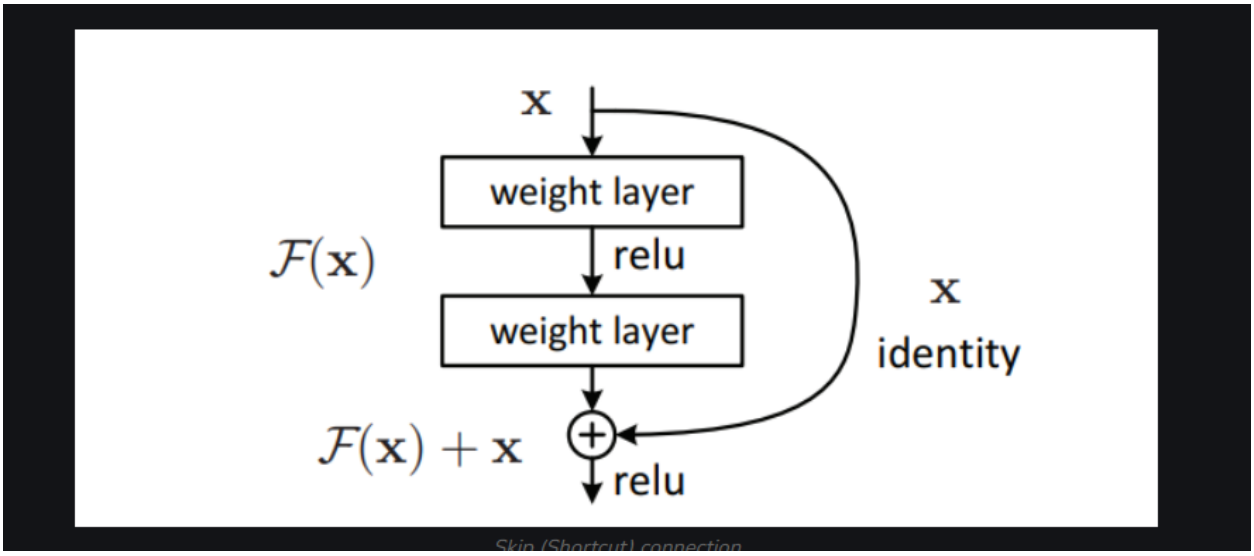

Figure 17. Skip Connections [25]

7.3 Driver Behavior Model Result

I tested all 3 networks to see which one would be the best. The accuracy of the networks was as follows from best to worst VGGNet (95.44 %), AlexNet (94.32%) and ResNet (33.16%).

I speculate that the reason ResNet performed the worst is because the implementation flattened the final feature maps into two dense layers of 4096 units rather than using the global average pooling layer that the original ResNet paper recommends. Combined with the use of binary cross-entropy loss in place of categorical cross-entropy on a five-class problem, this likely caused the network to fail to converge on the correct class boundaries. Re-training with categorical cross-entropy and global average pooling would be the natural next step.

7.4 Facial Verification Model Network

The facial verification model serves a different purpose to the behavior detection model. Rather than classifying what a driver is doing, it is responsible for identifying who the driver is. This is achieved using a Siamese Neural Network, a specialised architecture designed to determine whether two images belong to the same person. Unlike a standard classification model which learns to assign images to fixed categories, a Siamese network learns a similarity function. It is trained to output a high score when two images match and a low score when they do not match. This makes it particularly well suited to facial verification, where the goal is to confirm identity rather than classify from a fixed set of known faces.

Unlike the behavior detection model, this model was trained on the Raspberry Pi 5 using Jupyter notebook

7.4.1 Preprocessing

Each image was preprocessed before being passed into the model. The preprocessing function reads the image from disk, decodes it from JPEG format, resizes it to 100×100 pixels, and normalizes the pixel values to the 0–1 range:

```
def preprocess(file_path):
    #reads in image from the file_path in byte like format
    #byte like format is any object that supports the buffer protocol
    byte_img = tf.io.read_file(file_path)
    #Load/decode the image
    img = tf.io.decode_jpeg(byte_img)
    #resize the image to 100 by 100 by 3 array
    img = tf.image.resize(img,(100,100))
    #pixel value is between 0 and 255
    #Scaling down the pixel value down by 255 to be between 0 and 1S
    img = img / 255.0
    #returns the image
    return img
```

Figure 18. Preprocess - Scale and Resize

A twin preprocessing function was then applied across the entire dataset so that both images in each pair were preprocessed together:

```
#passes the input and validation image through preprocessing
def preprocess_twin(input_img, validation_img, label):
    return(preprocess(input_img),preprocess(validation_img),label)
```

```
# Preprocess
data = data.map(preprocess_twin)
data = data.cache()
data = data.shuffle(buffer_size=1024)
```

Figure 19. Preprocessing function

The dataset was then split 70/30 into training and test partitions, batched into groups of 16, and prefetched to improve pipeline efficiency:

```
#Training partition
train_data = data.take(round(len(data)*0.7))#takes the data and splits it for training data
train_data = train_data.batch(16)
train_data = train_data.prefetch(8)
```

Figure 20. Training Partition

```
#Testing partition
test_data = data.skip(round(len(data)*0.7))#skips the training data
test_data = test_data.take(round(len(data)*0.3))
```

Figure 21. Testing Partition

7.4.2 Siamese Network

The model is built around a Siamese architecture, which consists of three components: an embedding network, a distance layer, and a classification layer. [26]

The embedding network takes a single 100×100×3 image and maps it to a compact 4096-dimensional vector representation. Both the anchor image and the comparison image are passed through the same embedding network — sharing identical weights — so that their representations exist in the same learned feature space. The network uses four convolutional layers with progressively increasing filter sizes (64 → 128 → 128 → 256), each followed by max pooling, before flattening into a dense sigmoid output layer:

```
def make_embedding():
    inp = Input(shape=(100,100,3),name='input_image')#input layer

    c1 = Conv2D(64,(10,10), activation='relu')(inp)#64 filters @ 10x10 pixels
    m1 = MaxPooling2D(64, (2,2), padding='same')(c1)#64 filters and shape is 2x2

    c2 = Conv2D(128,(7,7), activation='relu')(m1)#128 filters @ 7x7 pixels
    m2 = MaxPooling2D(64, (2,2), padding='same')(c2)#64 filters and shape is 2x2

    c3 = Conv2D(128,(4, 4), activation='relu')(m2)#128 filters @ 4x4 pixels
    m3 = MaxPooling2D(64, (2,2), padding='same')(c3)#64 filters and shape is 2x2

    c4 = Conv2D(256,(4, 4), activation='relu')(m3)#256 filters @ 4x4 pixels
    f1 = Flatten()(c4)
    d1 = Dense(4096, activation='sigmoid')(f1)#4096 vectors

    return Model(inputs=inp, outputs=d1, name='embedding')#returns embedding
```

Figure 22. Embedding Layer

The distance layer is a custom Keras layer that computes the L1 distance (absolute difference) between the two embedding vectors. A small L1 distance means the two images are similar; a large distance means they are different:

```
class L1Dist(Layer):#passes base Layer class
    def __init__(self, **kwargs):#basic init method
        super().__init__()

    #standard Keras Siamese distance layer
    def call(self, input_embedding, validation_embedding):#input = anchor, validation = pos/neg img
        return tf.math.abs(input_embedding - validation_embedding)#similarity calculation
```

Figure 23. Distance Layer

The full Siamese model brings both components together. It takes two images as input, an anchor and a validation image then passes each through the shared embedding network, computes the L1 distance between their embeddings, and passes the result through a single sigmoid neuron to produce a match probability between 0 and 1:

```
def make_siamese_model():

    # Anchor image
    input_image = Input(name='input_img', shape=(100,100,3))

    # Validation image
    validation_image = Input(name='validation_img', shape=(100,100,3))

    # Combines siamese distance layer components
    siamese_layer = L1Dist()
    siamese_layer._name = 'distance'
    distances = siamese_layer(embeddingLayer(input_image), embeddingLayer(validation_image))

    # Classification layer
    classifier = Dense(1, activation='sigmoid')(distances)

    return Model(inputs=[input_image, validation_image], outputs=classifier, name='SiameseNetwork') # return siamese model
```

Figure 24. Siamese Model Code

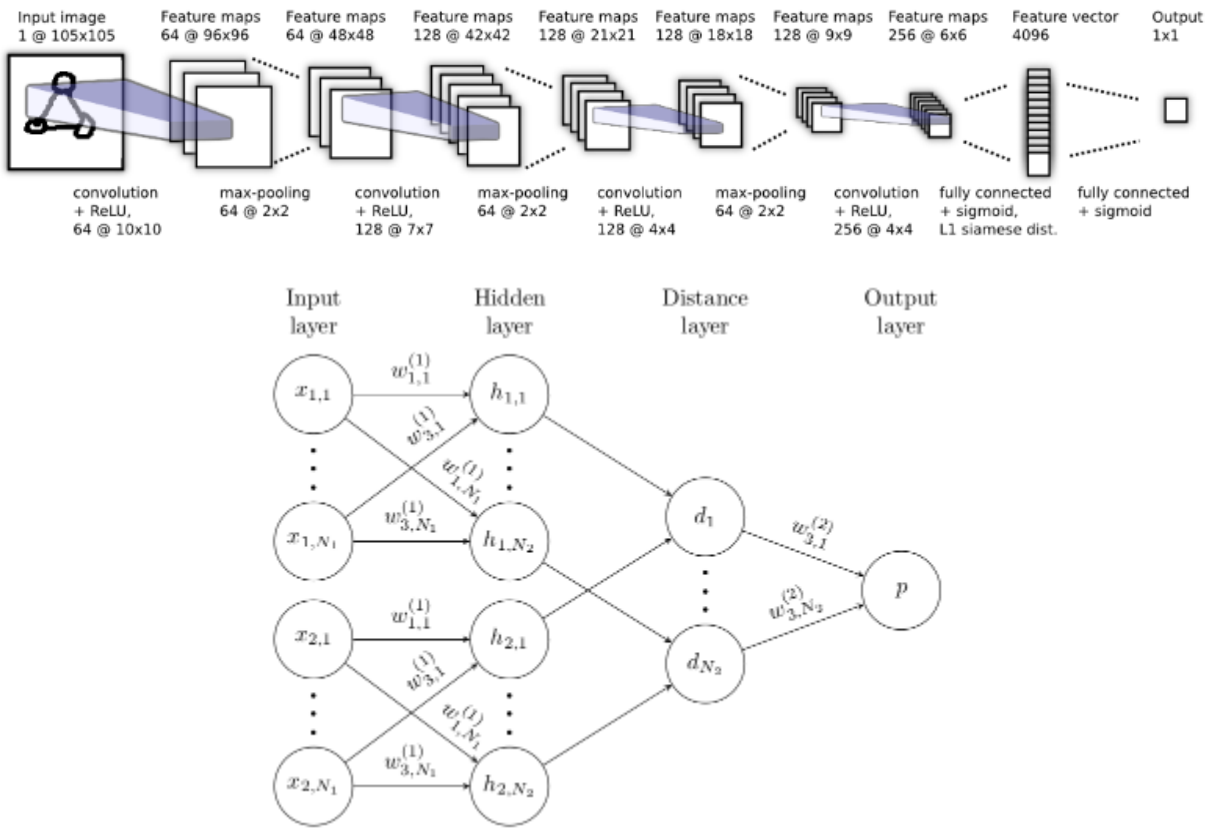

Figure 25. Siamese Network [26]

7.4.3 Training

The model was trained using a custom training loop rather than the standard Keras .fit() method, giving more fine-grained control over the gradient updates. Binary cross-entropy was used as the loss function, and the Adam optimizer was used with a learning rate of 1e-4. A GradientTape context was used to record operations for automatic differentiation, allowing gradients to be computed and applied manually each step:

```python
@tf.function #complies function into a callable tensorflow graph
def train_step(batch):
    with tf.GradientTape() as tape:#Records operations for automatic differentiation
        # Get anchor and positve/negative image
        X = batch[:2] #16 img per batch
        #get label
        y = batch[2]

        #Forward pass
        yhat = siamese_model(X, training=True)#perdiction
        #Calculate loss
        loss = binary_cross_loss(y, yhat)

    #Calculate gradients
    grad = tape.gradient(loss, siamese_model.trainable_variables)

    #Calculate updated weights and apply to siamese model
    opt.apply_gradients(zip(grad, siamese_model.trainable_variables))
    return loss
```

Figure 26. Training Step Function

The training loop ran for 50 epochs, with model checkpoints saved every 10 epochs to allow recovery of the best weights:

```python
def train(data, EPOCHS):
    # Loop through the epochs
    for epoch in range(1, EPOCHS+1):
        print('\n Epoch{}/{}'.format(epoch, EPOCHS))
        progbar = tf.keras.utils.Progbar(len(train_data))

        #Loop through each batch
        for idx, batch in enumerate(train_data):
            #Run training
            train_step(batch)
            progbar.update(idx+1)

        #Save checkpoints
        if epoch % 10 == 0:
            checkpoint.save(file_prefix=checkpoint_prefix)
```

Figure 27. Training Loop

7.5 Facial Verification Model Result

The trained model was evaluated on the held-out test set using precision and recall as the primary metrics. A prediction above 0.5 was treated as a positive match:

```
# Import metric calculations
from tensorflow.keras.metrics import Precision, Recall
```

```
test_input, test_val, y_true = test_data.as_numpy_iterator().next()
```

```
# Make predictions
y_hat = siamese_model.predict([test_input, test_val])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
```

```
# Post processing predictions
predictions = [1 if prediction > 0.5 else 0 for prediction in y_hat]
```

```
#Calculate Metrics
precision = Precision()
recall = Recall()
```

```
precision.update_state(y_true, y_hat)
recall.update_state(y_true, y_hat)
```

```
print(f'Precision: {precision.result().numpy()}')
print(f'Recall: {recall.result().numpy()}')
```

```
Precision: 1.0
Recall: 1.0
```

Figure 28. Precision Code

For real-world deployment, a `verify()` function was implemented that compares a live camera frame against a set of stored reference images. A detection threshold of 0.5 determines whether each individual comparison counts as a match, and a verification threshold of 0.5 determines what proportion of comparisons must match for the identity to be confirmed overall:

```python
def verify(model, detection_threshold, verification_threshold):
    # Build results array
    results = []

    # Get input image from verification folder
    for image in os.listdir(os.path.join('application_data', 'verification_images')):
        input_img = preprocess(os.path.join('application_data', 'verification_images', image))

        # Compare with images in input folder
        for val_image in os.listdir(os.path.join('application_data', 'input_image')):
            validation_img = preprocess(os.path.join('application_data', 'input_image', val_image))

            # Make predictions
            result = model.predict(list(np.expand_dims([input_img, validation_img], axis=1)))
            results.append(result)

    # Detection threshold: metric above which prediction is considered positive
    detection = np.sum(np.array(results) > detection_threshold)

    # Verification threshold: proportion of positive predictions / total positive samples
    verification = detection / len(os.listdir(os.path.join('application_data', 'verification_images')))
    verified = verification > verification_threshold

    return results, verified
```

Figure 29. Verify Function

This function was integrated with the Raspberry Pi Camera Module using picamera2 for real-time verification. Pressing v during the live camera feed triggers a verification check, with the result, VERIFIED or NOT VERIFIED overlaid directly onto the video frame in green or red respectively.

8 Website/Dashboard

With both machine learning models trained and validated, the next stage of the project was to wrap them in a usable application. The goal was to turn the raw camera feed and the two models into an end-to-end driver monitoring system: a driver sits in front of the Raspberry Pi, is recognized by the facial verification model, has their driving classified in real time by the behavior model, and sees their score reflected on a web dashboard that also feeds into a shared leaderboard.

The application is split across four cooperating components. The Raspberry Pi runs a Flask-based camera server that owns the camera, runs both models, and records behavior events. A Node.js Express server sits on the developer machine (or a separate host), handles user accounts, and talks to a MongoDB Atlas cluster. A Next.js frontend, also served from the same machine, provides the user interface. Finally, a small Node.js worker periodically syncs the Pi's local behavior log into MongoDB and recalculates the leaderboard. Each of these is described in turn below.

8.1 Camera Server

The camera server is implemented as a Python script, `camera_server.py`, that runs directly on the Raspberry Pi 5. It is the only component that talks to the Pi Camera Module 3 and the only component that runs the TensorFlow models in production. Every other part of the system interacts with the camera server over HTTP.

8.1.1 State Machine

A central design decision was to structure the server around an explicit three-state machine using the `transitions` Python library. The states are `unverified`, `verifying` and `verified` and the transitions between them are:

- `start_verify: unverified -> verifying`, triggered automatically on the first frame
- `verify_success: verifying -> verified`, triggered when the Siamese model confirms a match.

- `verify_fail: verifying -> unverified`, triggered when no match is found on a given frame.
- `reset: any state -> unverified`, triggered by the frontend when a user logs out or switches accounts.

This was preferred over ad-hoc Boolean flags because it makes the lifecycle of a session explicit: behavior classification only runs when the machine is in the verified state, which guarantees that no behavior events are recorded against an unknown identity. It also simplifies the frontend contract, which only need to poll a single `/status` endpoint to know exactly what the Pi is doing at any moment.

```
class VerificationStateMachine:
    states = ['unverified', 'verifying', 'verified']

    def __init__(self):
        self.verified_user = False
        self.detected_username = None
        self.expected_username = None
        self.current_frame = None
        self.lock = threading.Lock()
        self.frame_count = 0
        self.current_behavior = "Unknown"
        self.behavior_buffer = []
        self.last_recorded_behavior = None

        self.machine = Machine(model=self, states=VerificationStateMachine.states, initial='unverified')

        self.machine.add_transition('start_verify', 'unverified', 'verifying')
        self.machine.add_transition('verify_success', 'verifying', 'verified')
        self.machine.add_transition('verify_fail', 'verifying', 'unverified')
        self.machine.add_transition('reset', '*', 'unverified')
```

Figure 30. Class Verification State Machine

8.1.2 Frame Capture and Processing Loop

Capturing frames and running models at the same time on a constrained device is the main performance challenge. The server solves this with three cooperating threads. A detection loop thread owns the camera and produces frames at roughly 10 frames per second, matching the camera's configured frame rate. A second `_process_worker` thread is responsible for running the models on one frame at a time. A third thread is Flask's own HTTP server, which serves endpoints like `/stream` and `/frame`.

Communication between the capture loop and the processing worker is deliberately one-slot: the capture loop writes the latest frame into `_process_frame_pending` (overwriting any previous pending frame) and sets a `threading.Event`. The worker wakes, drains the single pending frame, clears the slot, and runs the models. If model inference is slower than capture which it often is for VGGNet, older frames are dropped rather than queued. This prevents thread accumulation and memory growth and keeps the system responsive on the Pi's limited resources.

```python
def _process_worker():
    #Single worker that processes one frame at a time — no thread accumulation.
    global _process_frame_pending
    while True:
        _process_frame_event.wait()
        _process_frame_event.clear()
        with _process_frame_lock:
            frame = _process_frame_pending
            _process_frame_pending = None
        if frame is not None:
            try:
                vsm.process_frame(frame)
            except Exception as e:
                print(f"[process_worker] ERROR: {e}")


def detection_loop():
    #Continuously capture frames; hand off to the single process worker.
    global latest_frame, _process_frame_pending
    print("Detection loop running.")
    loop_count = 0
    while True:
        try:
            frame = picam2.capture_array()
            with latest_frame_lock:
                latest_frame = frame.copy()
            with _process_frame_lock:
                _process_frame_pending = frame.copy()
            _process_frame_event.set()
            loop_count += 1
            if loop_count % 50 == 0:
                print(f"[detection_loop] alive — {loop_count} frames captured, state={vsm.state}, verified={vsm.verified_user}")
            time.sleep(0.1)  # ~10 fps, matches camera config
        except Exception as e:
            print(f"[detection_loop] ERROR: {e} — retrying in 2s")
            time.sleep(2)
```

Figure 31. _process_worker and detection_loop

8.1.3 Identity Verification

When the server is in the `verifying` state, every incoming frame is passed to `verify_frame`. The frame is first preprocessed into a 100x100 normalized tensor (same preprocessing used in training, see Section 7.5.1). The server then walks a directory of stored verification images on disk, one folder per registered user, and for each candidate user compares the live frame against every stored image using the loaded Siamese model. The per

image match threshold and per user verification threshold are both set to 0.5 in the deployed system. As soon as a user's match ratio exceeds the threshold, the state machine transitions to `verified` and the detected username is stored.

If the frontend has supplied an expected username via `/set-user` verification is scoped to that single folder rather than the full user directory. This is a meaningful optimization: on a Pi running TensorFlow, iterating through every stored face for every user is prohibitively slow, so constraining the search to the user who is trying to log in cuts verification time from seconds to a fraction of a second.

8.1.4 Behavior Classification and Debouncing

Once a user is verified, the server begins classifying driver behavior using the VGGNet model trained in Section 7.3.4. The VGGNet weights are loaded once at startup and reused for every prediction. To reduce load, classification is run on every other captured frame rather than every frame, giving an effective inference rate of around 5 Hz.

A naive system would record every predicted label directly — but this would produce an enormous and noisy log, since the classifier fluctuates between classes across adjacent frames. To smooth this out, the server maintains a rolling buffer of the last five predictions and will only commit an event when all five agree. In practice this means a driver must be consistently exhibiting a behavior for roughly one second before it is recorded, and the behavior must also differ from the last recorded behavior to avoid duplicate entries. The resulting signal is much cleaner: a typical drive produces a sequence of stable Safe / Turn / Texting events rather than a flood of per-frame noise.

Each recorded event is enriched with a points score drawn from a simple scoring table, Safe: +10, Turn: +5, Other: −2, Talking: −5, Texting: −10, and written to a local JSON log file on the Pi. In parallel, a best-effort HTTP POST is fired off to the Express server's /api/alerts endpoint on a background thread so that dangerous behaviors can appear on the dashboard almost immediately, independent of the slower 10-minute score sync described later.

```python
    def record_behavior(self, behavior):
        score = behavior_scores[behavior]
        record = {
            'username': self.detected_username,
            'behavior': behavior,
            'score': score,
            'timestamp': datetime.now().isoformat()
        }

        if os.path.exists(behavior_log_file):
            with open(behavior_log_file, 'r') as f:
                data = json.load(f)
        else:
            data = []

        data.append(record)
        with open(behavior_log_file, 'w') as f:
            json.dump(data, f, indent=2)

        print(f"✅ Recorded: {behavior} (score: {score}, user: {self.detected_username})")

        def post_alert():
            try:
                http_requests.post(
                    'http://localhost:5000/api/alerts',
                    json={
                        'username': self.detected_username,
                        'behavior': behavior,
                        'score': score,
                    },
                    timeout=2
                )
            except Exception:
                pass
        threading.Thread(target=post_alert, daemon=True).start()

```

Figure 32. Record Behavior Function

8.1.5 HTTP Endpoints

The camera server exposes a small, deliberately minimal HTTP API for the frontend to consume. `/stream` returns a multipart MJPEG stream, useful for debugging in a browser but not used by the production dashboard. `/frame` returns a single annotated JPEG of the most recent frame and is what the Live Feed page polls. `/status` returns the current verification state and behavior label, used by the dashboard as a lightweight heartbeat. `/verified-user` returns just the verified flag and detected username, used during face-based sign-in. `/set-user` lets the frontend tell the Pi which user is attempting to log in — with an optional verified: true flag that is used when the user has already authenticated via password, allowing the server to skip

facial verification and go straight to behavior classification. `/reset` clears the machine back to unverified.

CORS is configured explicitly to allow requests only from the Next.js frontend running on localhost:3000.

8.2 Backend API Server

The backend API server is a single server.js file built with Express 5 on Node.js. It runs on port 5000 and is responsible for everything that the camera server should not have to worry about: persistent user accounts, uploaded facial-setup videos, the leaderboard, and the historical behavior log as presented to the frontend. All persistent data lives in a MongoDB Atlas cluster reached via Mongoose.

8.2.1 Authentication

Two endpoints handle user accounts. POST `/register` accepts a username, email and password, hashes the password with `bcrypt` using 10 salt rounds, and inserts a new User document. The response distinguishes between a generic error and the specific case of a duplicate username or email (MongoDB error code 11000) so that the frontend can surface a helpful message. POST `/login` looks up the user by username, uses `bcrypt.compare` to verify the password against the stored hash, and returns either the username on success or a 401 on failure.

The system supports two sign-in paths, both of which ultimately drive the camera server's state machine. The first is a standard password login through POST `/login`; on success the frontend also calls the Pi's `/set-user` endpoint with verified: true to pre-verify the session. The second is a pure face-based sign-in, in which the frontend polls `/verified-user` on the Pi every two seconds and, as soon as the Siamese model identifies the user, treats that identity as authenticated. This second flow is convenient for a real vehicle, where typing a password is neither practical nor safe.

8.2.2 Facial Setup and Frame Extraction

When a user registers, they are redirected to a facial-setup page that accepts a short video of their face. The backend handles the upload at POST `/facial-setup` using the multer middleware, which saves the uploaded video to a temporary uploads/ directory. It then invokes ffmpeg as a subprocess with the filter fps=3 and a -frames:v 50 cap, extracting up to 50 still frames at three frames per second. The frames are written into application_data/verification_images/<username>/, which is the exact directory that the camera server reads from during verification. Once extraction completes, the temporary video file is deleted.

This design keeps the heavy training data entirely off-disk on the client side, the user only ever sees a short video being uploaded, while producing a solid pool of reference images for the Siamese model to compare against. A supporting GET `/facial-frames/:username` endpoint lets the frontend display how many frames are currently stored for a given user and DELETE `/facial-frames/:username` allows the user to clear and re-record their facial setup.

8.2.3 Behavior Logs, Alerts and Leaderboard

The remaining endpoints expose the data the dashboard needs. POST `/api/alerts` is the endpoint the camera server calls whenever it records a behavior event; the payload (username, behavior, score) is inserted into an Alert collection with a server-generated timestamp. GET `/api/alerts/:username` returns the fifty most recent alerts for a given user, which drives the Notifications page, and DELETE `/api/alerts/:username` lets the user clear them.

GET `/api/behavior-log?username=...` returns the full behavior history for a user, sorted chronologically, and is what the Behavior Dashboard page consumes to build its score chart and event log. GET `/api/leaderboard` returns all users ranked by cumulative score. A supplementary POST `/api/leaderboard/seed` endpoint was implemented to populate the leaderboard with twenty synthetic users during development, which made it possible to test the leaderboard UI before the system had collected enough real users to fill it.

GET `/api/userdatas/:userId` and POST `/api/userdatas/update` provide a simple per-day accumulator that records the net score change for each user on each calendar day. This provides the time-series data used in the dashboard's score-over-time chart.

8.2.4 Data Model

Persistent state lives in five MongoDB collections, each backed by a Mongoose schema defined at the top of server.js:

User stores the username, email and bcrypt-hashed password, with unique indexes on both username and email. Alert stores a username, behavior label, score and timestamp for each dangerous-behavior event, and is what drives the near-real-time Notifications page. BehaviorLog is the long-term history of every recorded event, keyed by username and timestamp; entries are mirrored here from the Pi's local log file by the sync worker. Leaderboard stores each user's cumulative score and current rank. UserData stores a per-user dailyData array of { date, value } pairs for charting. Using separate collections for real-time alerts and long-term history is deliberate: alerts are small, bounded and read frequently; the behavior log is append-only and read in bulk.

8.3 Score Sync Worker

The Pi and MongoDB are kept in step by a small Node.js script, sync_score.js, that runs on a ten-minute interval. On each tick it reads the Pi's local behavior_log.json, iterates through every entry, and upserts each one into the BehaviorLog collection keyed on the combination of username and timestamp. This makes the sync idempotent: running it repeatedly on the same log file produces no duplicates.

Once the log is mirrored, the worker groups all entries by username, sums the scores, and upserts each user's total into the Leaderboard collection. Finally it pulls the full leaderboard sorted by score descending and issues a single bulkWrite to reassign ranks 1..N across every user in one round-trip. Running the rank recalculation as a bulk operation rather than N individual updates is important for keeping the sync fast even as the user base grows.

The choice to sync every ten minutes rather than on every recorded event was a trade-off. Per-event writes to Atlas would give an instantly accurate leaderboard but would also mean that a transient network outage between the Pi and MongoDB would start dropping driving data. A batched approach, coupled with the Pi's durable local JSON log, means the system tolerates the backend being offline for extended periods. When it comes back, the next sync catches up automatically. The real-time /api/alerts channel covers the user-visible case where the driver needs to see a dangerous behavior acknowledged immediately.

8.4 Frontend

The frontend is a Next.js 14 application written in React. All pages live under src/pages/ and share a common layout that provides the sidebar, hamburger menu and global alert banner. The application uses Next.js's file-based routing, so each file under src/pages/ corresponds directly to a URL, for example src/pages/Leaderboard/index.js serves /Leaderboard.

8.4.1 Global State

Global state is managed through a single React Context provider, GlobalContextProvider, which holds the currently signed-in username along with a handful of UI flags. The username is persisted to localStorage on sign-in and re-hydrated on page load, which keeps the user signed in across refreshes without requiring a separate session mechanism. The context exposes an updateGlobals helper that every page uses to set the current user after a successful login, and a logout helper that clears both the local state and the stored username

8.4.2 Registration and Facial Setup

Registration is a two-step flow. The register.js page collects username, email and password and posts them to the backend's `/register` endpoint. On success the user is redirected directly to `/auth/facial-setup?username=...`, which renders the facial-setup.js page. This page shows short guidance on how to record a good setup video (good lighting, face the camera directly, no sunglasses), accepts an uploaded video via a standard HTML file input, and posts it as multipart/form-data to the backend's `/facial-setup` endpoint. The extracted frame count is displayed on success, after which the user can proceed to sign in. A "Skip for

now" button lets the user bypass facial setup, in which case they will need to sign in with their password.

8.4.3 Login

The login page offers both authentication methods on a single screen. A standard username/password form drives the `/login` endpoint; on success the global context is updated, and the camera server's `/set-user` is called with verified: true so that the Pi treats the session as pre-authenticated. Alternatively, the user can press "Sign in with Face Recognition", which starts a two-second polling loop against the camera server's `/verified-user` endpoint. As soon as the Siamese model reports a match, the detected username is written to the global context and the Pi is notified via `/set-user`, and the user is redirected to the dashboard. If the camera server is unreachable, polling stops and an error is displayed.

8.4.4 Live Feed

The Live Feed page is the visual confirmation that the system is working end-to-end. It polls the camera server's `/status` endpoint every three seconds to determine whether the Pi is online and what behavior it is currently classifying, and it polls `/frame?t=<timestamp>` every hundred milliseconds to refresh the displayed image. The `t=<timestamp>` query string is used purely as a cache-buster, without it the browser would serve the same JPEG from its cache on every request.

The current behavior is displayed as a colored pill above the stream, with colors chosen to make the classification obvious at a glance (green for safe, red for texting, orange for talking, blue for turning, amber for other). If the camera server cannot be reached, an offline card is shown instead of the stream, along with the exact command to start the server on the Pi (python camera_server.py) and a retry button. This last detail proved useful during development, since the Pi was occasionally power-cycled and the dashboard otherwise gave no indication of where the problem was.

8.4.5 Behavior Dashboard

The Behavior Dashboard, served at `/data`, is where the user sees their historical driving record. On mount it pulls the full behavior log from `/api/behavior-log?username=...` and refreshes it every five seconds.

The page is composed of three parts. At the top, an inline SVG line chart plots the user's cumulative score over time, computed by walking the behavior entries in chronological order and accumulating the per-event scores. Implementing the chart directly in SVG rather than pulling in a charting library kept the dependency footprint small and made it easy to match the exact visual style of the rest of the dashboard. Below the chart, a row of colored summary cards shows the cumulative score, total event count, and a per-behavior tally. Finally, a paginated event log at the bottom lists every recorded behavior with its score and timestamp, twenty entries per page.

8.4.6 Notifications

The Notifications page is a focused view of dangerous-behavior alerts. It fetches the last fifty alerts for the current user from `/api/alerts/:username` every five seconds and displays each one as a colored card. The human-readable message (for example "Driver was texting" or "Driver was talking on the phone") is derived from the raw behavior label via a small lookup table, and the card's left border is color-coded by severity. A "Clear All" button calls the DELETE endpoint to wipe the user's alerts when they have reviewed them.

8.4.7 Leaderboard

The Leaderboard page fetches the full ranking from `/api/leaderboard` and renders a styled list. The top three positions are highlighted with gold, silver and bronze medal emojis; every subsequent row shows the numeric rank. Scores are color-coded into three bands — high (≥ 800), medium (≥ 500) and low — to give a quick visual sense of who is driving well and who is not. If the current user appears in the list, their row is highlighted and flagged with a "You" badge, so the user can immediately see where they stand. This gamification layer was the motivating application feature of the project: everything the Pi records ultimately shows up here as either a rise or a fall in the driver's standing against everyone else.

9 Testing

To validate my project, I tested the web application. These tests were as follows, user testing, performance testing and unit tests.

9.1 User Testing

For User Testing I let my brother make an account on the web app and see what score he got after 10 minutes. This test allowed me to see if there was anything to improve with the user experience. It also allowed me to see that the behavior model is not 100% accurate all the time.

9.2 Performance Testing

I used Prometheus and NetData to read the performance results of the Raspberry Pi 5. When running the camera server and sync_score file for 30 minutes the max temperature reached 85 degrees Celsius. At this temperature the Pi is throttled and could potentially crash.

Figure 33. Thermal Throttling

9.3 Unit Tests

I used Playwright to perform the unit test.

To validate the RoadRank web application, end-to-end (E2E) testing was implemented using Playwright, an open-source browser automation framework developed by Microsoft. Unlike unit testing frameworks that isolate individual functions, Playwright drives a real Chromium browser instance against the running application, allowing tests to simulate genuine user interactions such as filling forms, clicking buttons, and navigating between pages. This approach

was chosen over alternatives such as Cypress due to Playwright's faster parallel test execution, built-in network interception capabilities, and its ability to auto-start the Next.js development server before the test suite runs, removing any manual setup requirement.

A total of 33 tests were written across three test files, home.spec.js, auth.spec.js, and leaderboard.spec.js each targeting a distinct area of the application.

Of the 33 tests executed, 33 passed following correction of selector ambiguities and route pattern specificity issues encountered during the initial run. All tests execute in parallel across a headless Chromium browser and complete in under two minutes. An HTML report is generated automatically after each run via npx playwright show-report, providing a visual breakdown of each test's result, duration, and any failure screenshots.

Search tests | All 33 | Passed 33 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium — 4/25/2026, 9:12:54 PM Total time: 10.4s

auth.spec.js

- Login Page › renders login form fields (chromium) — auth.spec.js:8 — 1.4s
- Login Page › shows face recognition button (chromium) — auth.spec.js:17 — 1.4s
- Login Page › shows error message on invalid credentials (chromium) — auth.spec.js:22 — 1.6s
- Login Page › shows network error when server is unreachable (chromium) — auth.spec.js:34 — 1.6s
- Login Page › "Create one" link navigates to register page (chromium) — auth.spec.js:44 — 1.1s
- Login Page › face recognition button shows scanning state (chromium) — auth.spec.js:50 — 728ms
- Login Page › Cancel button stops face scanning and restores button (chromium) — auth.spec.js:58 — 698ms
- Login Page › successful login redirects to home page (chromium) — auth.spec.js:67 — 1.0s
- Login Page › Sign In button shows loading state while submitting (chromium) — auth.spec.js:85 — 840ms
- Register Page › renders register form fields (chromium) — auth.spec.js:103 — 606ms
- Register Page › "Sign in" link navigates to login page (chromium) — auth.spec.js:112 — 664ms
- Register Page › shows error when username already exists (chromium) — auth.spec.js:118 — 660ms
- Register Page › shows network error when server is unreachable (chromium) — auth.spec.js:130 — 802ms
- Register Page › successful registration redirects to facial-setup (chromium) — auth.spec.js:140 — 1.6s
- Register Page › Create Account button shows loading state while submitting (chromium) — auth.spec.js:152 — 962ms
- Protected Route Redirect › visiting /data while logged out redirects to login (chromium) — auth.spec.js:167 — 1.3s

home.spec.js

- Home Page › shows hero title (chromium) — home.spec.js:4 — 882ms
- Home Page › shows Sign In and Create Account buttons when logged out (chromium) — home.spec.js:9 — 483ms
- Home Page › Sign In button navigates to login page (chromium) — home.spec.js:15 — 677ms
- Home Page › Create Account button navigates to register page (chromium) — home.spec.js:21 — 639ms
- Home Page › shows all six feature cards (chromium) — home.spec.js:27 — 594ms
- Home Page › shows "Everything you need" section heading (chromium) — home.spec.js:36 — 533ms
- Home Page › shows View Live Feed and My Dashboard buttons when logged in (chromium) — home.spec.js:41 — 1.1s

leaderboard.spec.js

- Leaderboard Page › renders page title (chromium) — leaderboard.spec.js:16 — 698ms
- Leaderboard Page › shows all players by name (chromium) — leaderboard.spec.js:22 — 696ms
- Leaderboard Page › shows medal emojis for top 3 (chromium) — leaderboard.spec.js:30 — 644ms
- Leaderboard Page › shows rank number for players outside top 3 (chromium) — leaderboard.spec.js:38 — 604ms
- Leaderboard Page › shows player scores (chromium) — leaderboard.spec.js:44 — 587ms
- Leaderboard Page › highlights current user with "You" badge (chromium) — leaderboard.spec.js:52 — 879ms
- Leaderboard Page › shows error message when API fails (chromium) — leaderboard.spec.js:61 — 584ms
- Leaderboard Page › shows Retry button when API fails (chromium) — leaderboard.spec.js:69 — 557ms
- Leaderboard Page › Retry button refetches and shows data on success (chromium) — leaderboard.spec.js:77 — 738ms
- Leaderboard Page › shows empty list when API returns no players (chromium) — leaderboard.spec.js:96 — 452ms

Figure 34. Playwright Tests

10 Ethics

A driver monitoring system that captures continuous video, performs biometric identification and stores behavioral records about a named individual sits in a sensitive part of the technology landscape and the ethical implications deserve to be examined.

10.1 Privacy and Biometric Data

The most significant technical concern is the use of facial verification. The Siamese neural network described in Chapter 7 generates a 4096 dimensional embedding of the driver's face, which is stored on the server to confirm the identity of the user during subsequent drives. Under GDPR (General Data Protection Regulation), facial recognition data is classified as biometric special category data and is subject to stricter handling requirements than ordinary personal data. The system's design tries to respect this in several ways. The user is not required to explicitly enrol their face during registration, so identification is opt-in rather than passive surveillance. Only the embedding is retained for verification purposes, the original enrolment frames are discarded after the model has computed the embedding. This limits the usefulness of the stored data outside its intended purpose. The MongoDB Atlas cluster that holds user records is access-controlled and the credentials are not exposed to the front-end. In a production deployment the embeddings would be encrypted at rest, and a clear retention policy with a self-service deletion endpoint would be added; for the prototype, deletion is performed manually.

10.2 Driver Consent and Third Parties

The system was designed to be used by a single, identified driver who has actively chosen to enrol. This consent model works for a personal use product, but it does not extend to passengers who might be visible to the camera. Although the camera is positioned to face the driver and frames are processed only for the verified user, it is realistic that other people will sometimes appear in the field of view. The current implementation does not store full frames for behavior classification which significantly reduces the risk of capturing third-party biometric data. Even so, a deployed version of the system would benefit from a clearly visible notice inside the vehicle informing passengers that a driver monitoring camera is in use.

10.3 Bias in Machine Learning

The behavior model was trained on the REVITSONE 5-Class Driver Behavior Dataset, which contains imagery from a varied but ultimately finite set of drivers and lighting conditions. As is well established in the machine learning literature, models trained on such datasets can perform unevenly across demographic groups that are underrepresented in the training data.

In the context of RoadRank, an under performing model could lead to a particular driver being unfairly penalised in the gamified scoring system.

10.4 Gamification and Behavioral Effects

The decision to attach scores to driving behaviors, and to display those scores on a public leaderboard, introduces ethical considerations of its own. Gamified safety systems can be effective at encouraging better behavior, but they can also incentivize users to *appear* safe to the camera rather than to *be* safe on the road, or to be reluctant to use the system at all if they fear social comparison.

10.5 Potential Misuse

A driver-monitoring product, even one designed for personal use, can be repurposed for surveillance by parties such as insurance companies, fleet operators or family members. *RoadRank* was therefore scoped as a tool that the driver controls; only the registered user can view their own score history and alerts, and there is no administrator role in the system that would allow a third party to view another user's data.

11 Conclusion

The aim of this project was to build a working driver-monitoring system that uses machine learning to recognize the driver, classify their driving behavior in real time, and present the results back to them through a usable web application. That aim has been met. RoadRank is delivered as a complete end-to-end product: a Raspberry Pi 5 running two trained models, an Express API backed by MongoDB Atlas, a Next.js dashboard, and a small sync worker keeping the leaderboard consistent.

The web application was built around the requirements that emerged once the models were running. Driver registration, facial enrolment, dual-mode authentication, a live camera feed, a paginated event log, a notifications page and a global leaderboard were all implemented and validated by a 33-test Playwright suite that runs in under two minutes against a real Chromium browser. Performance under load was monitored with Prometheus and NetData, which surfaced the thermal headroom of the Pi 5 during sustained use as a constraint that shaped the final architecture.

Reflecting on the project, the most striking lesson was how much application engineering surrounded the machine learning. The two models, despite being the conceptual center of the project, accounted for perhaps 40% of the total effort; the state machine on the Pi, the threading model that kept inference from blocking capture, the debouncing logic that turned noisy per-frame predictions into a clean event timeline, and the sync worker that traded instant leaderboard accuracy for resilience to network outages. None of these problems were visible during training; they only appeared once the models were running on the device.

Several limitations point toward future work. The behavior model was trained on a single public dataset and would benefit from fine-tuning on a more drivers to reduce the bias risk discussed in Chapter 11. On the hardware side, migrating the camera server to a Jetson-class device would relieve the thermal pressure observed during the performance tests and allow per-frame classification rather than the current debouncing window. On the application side, replacing the localStorage-based session with JWT authentication, encrypting the stored embeddings at rest,

and exposing a self-service deletion endpoint would bring the prototype closer to a deployable product.

12 Appendix

References

[1] RSA, "Road Deaths in 2023 Increase by 19%," 1 1 2024. [Online]. Available: https://www.rsa.ie/news-events/news/details/2024/01/01/road-deaths-in-2023-increase-by-19. [Accessed 2026].

[2] RSA, "road-deaths-in-2024-drop-by-4," 1 1 2025. [Online]. Available: https://www.rsa.ie/news-events/news/details/2025/01/01/road-deaths-in-2024-drop-by-4. [Accessed 2026].

[3] RSA, "Road Safety Authority | Annual Report 2023 - RSA," 2024. [Online]. Available: https://www.rsa.ie/docs/default-source/road-safety/annual-report-2023/rsa-annual-report-2023-.pdf?sfvrsn=a79116f6_5. [Accessed 2026].

[4] S. Burns, "Number of road deaths so far this year passes total figure for 2024," The Irish Times, 15 December 2025. [Online]. Available: https://www.irishtimes.com/ireland/2025/12/15/number-of-road-deaths-so-far-this-year-passes-total-figure-for-2024/. [Accessed 2026].

[5] Drinkaware, "Drink Driving in Ireland | Drink Driving Laws in Ireland | Drinkaware," 2 8 2024. [Online]. Available: https://drinkaware.ie/facts/drink-driving-in-ireland/. [Accessed 2026].

[6] L. Murray, "Ireland ranked seventh in Europe for number of road deaths in 2024 amid ‘alarming rise," The Irish Times, 24 7 2025. [Online]. Available: https://www.irishtimes.com/ireland/2025/07/24/ireland-ranked-seventh-in-europe-for-number-of-road-deaths-in-2024-amid-alarming-rise/. [Accessed 2026].

[7] S. S. a. R. J. H. D. Minott, "Benchmarking Edge AI Platforms: Performance Analysis of NVIDIA Jetson and Raspberry Pi 5 with Coral TPU," March 2025. [Online]. Available: 10.1109/SoutheastCon56624.2025.10971592.

[8] raspberrypi.com, "www.raspberrypi.com," [Online]. Available: https://www.raspberrypi.com/software/.

[9] python.org, "www.python.org," [Online]. Available: https://www.python.org/doc/essays/blurb/.

[10] Team Vast, "google collab explained simplifying your workflow with cloud tools," Vast, [Online]. Available: https://vast.ai/article/google-collab-explained-simplifying-your-workflow-with-cloud-tools.

[11] https://opencv.org, "about," opencv, [Online]. Available: https://opencv.org/about/.

[12] flask.palletsprojects, "stable," flask, [Online]. Available: https://flask.palletsprojects.com/en/stable/.

[13] www.w3schools, "React Introduction," w3schools, [Online]. Available: https://www.w3schools.com/React/react_intro.asp.

[14] nextjs.org, "docs," nextjs, [Online]. Available: https://nextjs.org/docs.

[15] Coursera, "What Is Node.js? Understanding Its Applications," [Online]. Available: https://www.coursera.org/articles/node-js.

[16] codecademy, "Express.js Explained: What it is and How to Use it in Your JavaScript Project," [Online]. Available: https://www.codecademy.com/article/what-is-express-js.

[17] IBM, "What is MongoDB?," IBM, [Online]. Available: https://www.ibm.com/think/topics/mongodb.

[18] L. E. B. Y. B. a. P. H. Y. Lecun, "Gradient-Based Learning Applied to Document Recognition," 1998. [Online]. Available: http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf.

[19] Stanford, "CS231n Deep Learning For Computer Vision," [Online]. Available: https://cs231n.github.io/convolutional-networks/.

[20] A. Jain, "All about convolutions, kernels, features in CNN - Abhishek Jain - Medium," 12 February 2024. [Online]. Available: https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1.

[21] A. Rosebrock, "Convolutional Neural Networks (CNNs) and Layer Types," 14 May 2021. [Online]. Available: https://pyimagesearch.com/2021/05/14/convolutional-neural-networks-cnns-and-layer-types/.

[22] J. C. Luna, "A Beginner’s Guide to the Rectified Linear Unit (ReLU)," 28 January 2025. [Online]. Available: https://www.datacamp.com/blog/rectified-linear-unit-relu.

[23] İ. Kılıç, "What is Max Pooling and Why Do We Need Max Pooling?," 3 September 2023. [Online]. Available: https://medium.com/@ilyurek/what-is-max-pooling-and-why-do-we-need-max-pooling-57247a3fbca9.

[24] geeksforgeeks, "What is Fully Connected Layer in Deep Learning," 16 April 2026. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/what-is-fully-connected-layer-in-deep-learning/.

[25] GeeksforGeeks, "GeeksforGeeks," 3 June 2020. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/.

[26] R. Z. a. R. S. G. Koch, "Siamese Neural Networks for One-shot Image Recognition," 2015. [Online]. Available: https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf.

[27] [Online].

[28] H. Kinsley, "Reinforcement Learning," PythonProgramming, [Online]. Available: https://pythonprogramming.net/q-learning-reinforcement-learning-python-tutorial/. [Accessed 02 02 2021].

[29] MakeSigns, "Scientic Posters Tutorial," [Online]. Available: https://www.makesigns.com/tutorials/scientific-poster-parts.aspx. [Accessed 09 02 2021].

[30] [Online].

[31] Arduino. [Online]. Available: https://www.arduino.cc/. [Accessed 09 02 2021].